Clinic Management System — Full Implementation Guide

(Updated: Sanctum → Laravel Passport · Missing Implementations Added · Docker Added)

Phase 1 — Project Initialization

Step 1: Scaffold the project

```bash
laravel new clinic-api --api
cd clinic-api
```

Step 2: Configure `.env`

```env
APP_NAME=ClinicAPI
APP_ENV=local
APP_KEY=           # php artisan key:generate
APP_DEBUG=true
APP_URL=http://localhost

DB_CONNECTION=mysql
DB_HOST=db          # Docker service name (use 127.0.0.1 for local)
DB_PORT=3306
DB_DATABASE=clinic_api
DB_USERNAME=clinic_user
DB_PASSWORD=secret

CACHE_DRIVER=redis
QUEUE_CONNECTION=redis
SESSION_DRIVER=redis

REDIS_HOST=redis    # Docker service name (use 127.0.0.1 for local)
REDIS_PASSWORD=null
REDIS_PORT=6379

PASSPORT_PERSONAL_ACCESS_CLIENT_ID=
PASSPORT_PERSONAL_ACCESS_CLIENT_SECRET=
```

Step 3: Install Laravel Passport

```bash
```

```
composer require laravel/passport
php artisan install:api --passport
```

`install:api --passport` publishes the migrations, adds the `api` guard, and scaffolds `config/auth.php`.

Step 4: Publish and run Passport migrations, then install keys

```bash
php artisan migrate
php artisan passport:install
```

`passport:install` creates the encryption keys and seeds the `oauth_clients` table. Note the **Personal Access Client ID and Secret** printed to the console — copy them into your `.env`.

Step 5: Register Passport routes and middleware in `bootstrap/app.php`

```php
use Laravel\Passport\Http\Middleware\CheckForAnyScope;
use Laravel\Passport\Http\Middleware\CheckScopes;

->withMiddleware(function (Middleware $middleware) {
    $middleware->alias([
        'role'         => \App\Http\Middleware\RoleMiddleware::class,
        'clinic.scope' => \App\Http\Middleware\ClinicScopeMiddleware::class,
        'scopes'       => CheckScopes::class,
        'scope'        => CheckForAnyScope::class,
    ]);
})
```

Step 6: Update `config/auth.php` — set the `api` guard to use Passport

```php
```

```
'guards' => [
    'web' => [
        'driver'   => 'session',
        'provider' => 'users',
    ],

    // Passport-powered guard — replaces Sanctum
    'api' => [
        'driver'   => 'passport',
        'provider' => 'users',
    ],
],
```

Phase 2 — Enums

app/Enums/UserRole.php

```php
<?php

namespace App\Enums;

enum UserRole: string
{
    case SuperAdmin = 'super_admin';
    case Doctor     = 'doctor';
    case Assistant  = 'assistant';
}
```

app/Enums/AppointmentStatus.php

```php
```

```php
<?php

namespace App\Enums;

enum AppointmentStatus: string
{
    case Pending   = 'pending';
    case Confirmed = 'confirmed';
    case Cancelled = 'cancelled';
    case Completed = 'completed';
}
```

`app/Enums/Gender.php`

```php
<?php

namespace App\Enums;

enum Gender: string
{
    case Male   = 'male';
    case Female = 'female';
}
```

Phase 3 — Migrations (run in this exact order)

Step 1: `create_clinics_table`

```php
Schema::create('clinics', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('address')->nullable();
    $table->string('phone')->nullable();
    $table->string('email')->nullable();
    $table->boolean('is_active')->default(true);
    $table->timestamps();
    $table->softDeletes(); // added for safe deletion
});
```

Step 2: Modify default `create_users_table`

```php
Schema::create('users', function (Blueprint $table) {
    $table->id();
    $table->foreignId('clinic_id')->nullable()->constrained()->nullOnDelete();
    $table->string('name');
    $table->string('email')->unique();
    $table->string('password');
    $table->string('phone')->nullable();
    $table->enum('role', ['super_admin', 'doctor', 'assistant']);
    $table->boolean('is_active')->default(true);
    $table->timestamps();
});
```

Step 3: `create_patients_table`

```php
Schema::create('patients', function (Blueprint $table) {
    $table->id();
    $table->foreignId('clinic_id')->constrained()->cascadeOnDelete();
    $table->string('name');
    $table->string('email')->nullable();
    $table->string('phone');
    $table->date('date_of_birth')->nullable();
    $table->enum('gender', ['male', 'female']);
    $table->text('address')->nullable();
    $table->timestamps();
    $table->softDeletes();
});
```

Step 4: `create_doctor_schedules_table`

```php
```

```php
Schema::create('doctor_schedules', function (Blueprint $table) {
    $table->id();
    $table->foreignId('doctor_id')->constrained('users')->cascadeOnDelete();
    $table->foreignId('clinic_id')->constrained()->cascadeOnDelete();
    $table->tinyInteger('day_of_week'); // 0=Sunday … 6=Saturday
    $table->time('start_time');
    $table->time('end_time');
    $table->tinyInteger('slot_duration')->default(30); // minutes
    $table->boolean('is_active')->default(true);
    $table->timestamps();

    $table->unique(['doctor_id', 'day_of_week']); // one schedule per day per d
});
```

Step 5: `create_appointments_table`

```php
Schema::create('appointments', function (Blueprint $table) {
    $table->id();
    $table->foreignId('clinic_id')->constrained()->cascadeOnDelete();
    $table->foreignId('doctor_id')->constrained('users');
    $table->foreignId('patient_id')->constrained('patients');
    $table->foreignId('booked_by')->constrained('users');
    $table->date('appointment_date');
    $table->time('start_time');
    $table->time('end_time');
    $table->enum('status', ['pending', 'confirmed', 'cancelled', 'completed'])-
    $table->text('notes')->nullable();
    $table->timestamps();

    // Prevents double-booking at DB level
    $table->unique(['doctor_id', 'appointment_date', 'start_time']);
});
```

Step 6: `create_prescriptions_table`

```php
```

```php
Schema::create('prescriptions', function (Blueprint $table) {
    $table->id();
    $table->foreignId('appointment_id')->constrained()->cascadeOnDelete();
    $table->foreignId('doctor_id')->constrained('users');
    $table->foreignId('patient_id')->constrained('patients');
    $table->foreignId('clinic_id')->constrained();
    $table->text('diagnosis');
    $table->text('notes')->nullable();
    $table->timestamps();
});
```

Step 7: `create_prescription_items_table`

```php
Schema::create('prescription_items', function (Blueprint $table) {
    $table->id();
    $table->foreignId('prescription_id')->constrained()->cascadeOnDelete();
    $table->string('medicine_name');
    $table->string('dosage');
    $table->string('frequency');
    $table->string('duration');
    $table->text('notes')->nullable();
    $table->timestamps();
});
```

Phase 4 — Models

`app/Models/Clinic.php`

```php
```

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\SoftDeletes;

class Clinic extends Model
{
    use SoftDeletes;

    protected $fillable = ['name', 'address', 'phone', 'email', 'is_active'];

    protected $casts = ['is_active' => 'boolean'];

    public function users()
    {
        return $this->hasMany(User::class);
    }

    public function patients()
    {
        return $this->hasMany(Patient::class);
    }

    public function appointments()
    {
        return $this->hasMany(Appointment::class);
    }
}
```

`app/Models/User.php`

> **Passport change:** `HasApiTokens` is now imported from `Laravel\Passport` instead of `Laravel\Sanctum`.

```php
```

```php
<?php

namespace App\Models;

use App\Enums\UserRole;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Laravel\Passport\HasApiTokens; // ← Passport, NOT Sanctum

class User extends Authenticatable
{
    use HasApiTokens;

    protected $fillable = ['clinic_id', 'name', 'email', 'password', 'phone', '

    protected $hidden = ['password', 'remember_token'];

    protected $casts = [
        'role'      => UserRole::class,
        'is_active' => 'boolean',
    ];

    public function clinic()
    {
        return $this->belongsTo(Clinic::class);
    }

    public function schedules()
    {
        return $this->hasMany(DoctorSchedule::class, 'doctor_id');
    }

    public function appointments()
    {
        return $this->hasMany(Appointment::class, 'doctor_id');
    }

    public function prescriptions()
    {
        return $this->hasMany(Prescription::class, 'doctor_id');
    }
}
```

app/Models/Patient.php

```php
```

```php
<?php

namespace App\Models;

use App\Enums\Gender;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\SoftDeletes;

class Patient extends Model
{
    use SoftDeletes;

    protected $fillable = ['clinic_id', 'name', 'email', 'phone', 'date_of_birt

    protected $casts = [
        'gender'        => Gender::class,
        'date_of_birth' => 'date',
    ];

    public function clinic()
    {
        return $this->belongsTo(Clinic::class);
    }

    public function appointments()
    {
        return $this->hasMany(Appointment::class);
    }

    public function prescriptions()
    {
        return $this->hasMany(Prescription::class);
    }
}
```

app/Models/DoctorSchedule.php

```php
```

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class DoctorSchedule extends Model
{
    protected $fillable = [
        'doctor_id', 'clinic_id', 'day_of_week',
        'start_time', 'end_time', 'slot_duration', 'is_active',
    ];

    protected $casts = ['is_active' => 'boolean'];

    public function doctor()
    {
        return $this->belongsTo(User::class, 'doctor_id');
    }

    public function clinic()
    {
        return $this->belongsTo(Clinic::class);
    }
}
```

app/Models/Appointment.php

```php
```

```php
<?php

namespace App\Models;

use App\Enums\AppointmentStatus;
use Illuminate\Database\Eloquent\Model;

class Appointment extends Model
{
    protected $fillable = [
        'clinic_id', 'doctor_id', 'patient_id', 'booked_by',
        'appointment_date', 'start_time', 'end_time', 'status', 'notes',
    ];

    protected $casts = [
        'status'           => AppointmentStatus::class,
        'appointment_date' => 'date',
    ];

    public function doctor()
    {
        return $this->belongsTo(User::class, 'doctor_id');
    }

    public function patient()
    {
        return $this->belongsTo(Patient::class);
    }

    public function clinic()
    {
        return $this->belongsTo(Clinic::class);
    }

    public function bookedBy()
    {
        return $this->belongsTo(User::class, 'booked_by');
    }

    public function prescription()
    {
        return $this->hasOne(Prescription::class);
    }
}
```

app/Models/Prescription.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Prescription extends Model
{
    protected $fillable = ['appointment_id', 'doctor_id', 'patient_id', 'clinic

    public function appointment()
    {
        return $this->belongsTo(Appointment::class);
    }

    public function items()
    {
        return $this->hasMany(PrescriptionItem::class);
    }

    public function patient()
    {
        return $this->belongsTo(Patient::class);
    }

    public function doctor()
    {
        return $this->belongsTo(User::class, 'doctor_id');
    }

    public function clinic()
    {
        return $this->belongsTo(Clinic::class);
    }
}
```

app/Models/PrescriptionItem.php

```php
```

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class PrescriptionItem extends Model
{
    protected $fillable = ['prescription_id', 'medicine_name', 'dosage', 'frequ

    public function prescription()
    {
        return $this->belongsTo(Prescription::class);
    }
}
```

Phase 5 — Middleware

app/Http/Middleware/RoleMiddleware.php

```php
```

```php
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class RoleMiddleware
{
    public function handle(Request $request, Closure $next, string ...$roles):
    {
        $user = $request->user();

        if (! $user || ! in_array($user->role->value, $roles)) {
            return response()->json([
                'success' => false,
                'message' => 'Forbidden: insufficient role.',
            ], 403);
        }

        return $next($request);
    }
}
```

app/Http/Middleware/ClinicScopeMiddleware.php

```php
```

```php
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;

class ClinicScopeMiddleware
{
    public function handle(Request $request, Closure $next)
    {
        $user = $request->user();

        if ($user->role->value === 'super_admin') {
            return $next($request);
        }

        if (! $user->is_active || ! $user->clinic || ! $user->clinic->is_active
            return response()->json([
                'success' => false,
                'message' => 'Your account or clinic is inactive.',
            ], 403);
        }

        $request->merge(['clinic_id' => $user->clinic_id]);

        return $next($request);
    }
}
```

Phase 6 — Repositories

app/Repositories/ClinicRepository.php

```php
```

```php
<?php

namespace App\Repositories;

use App\Models\Clinic;

class ClinicRepository
{
    public function all(int $perPage = 15)
    {
        return Clinic::paginate($perPage);
    }

    public function find(int $id): Clinic
    {
        return Clinic::with('users')->findOrFail($id);
    }

    public function create(array $data): Clinic
    {
        return Clinic::create($data);
    }

    public function update(Clinic $clinic, array $data): Clinic
    {
        $clinic->update($data);
        return $clinic;
    }

    public function toggle(Clinic $clinic): Clinic
    {
        $clinic->update(['is_active' => ! $clinic->is_active]);
        return $clinic;
    }

    public function delete(Clinic $clinic): void
    {
        $clinic->delete(); // soft delete
    }
}
```

app/Repositories/UserRepository.php

```php
```

```php
<?php

namespace App\Repositories;

use App\Models\User;

class UserRepository
{
    public function all(array $filters = [], int $perPage = 15)
    {
        return User::with('clinic')
            ->when(isset($filters['role']),      fn($q) => $q->where('role', $f
            ->when(isset($filters['clinic_id']), fn($q) => $q->where('clinic_id
            ->paginate($perPage);
    }

    public function find(int $id): User
    {
        return User::with('clinic')->findOrFail($id);
    }

    public function create(array $data): User
    {
        return User::create($data);
    }

    public function update(User $user, array $data): User
    {
        $user->update($data);
        return $user;
    }

    public function toggle(User $user): User
    {
        $user->update(['is_active' => ! $user->is_active]);
        return $user;
    }
}
```

app/Repositories/PatientRepository.php

```php
```

```php
<?php

namespace App\Repositories;

use App\Models\Patient;

class PatientRepository
{
    public function allForClinic(int $clinicId, ?string $search = null, int $pe
    {
        return Patient::where('clinic_id', $clinicId)
            ->when($search, fn($q) => $q->where(function ($q) use ($search) {
                $q->where('name', 'like', "%{$search}%")
                  ->orWhere('phone', 'like', "%{$search}%")
                  ->orWhere('email', 'like', "%{$search}%");
            }))
            ->paginate($perPage);
    }

    public function find(int $id): Patient
    {
        return Patient::with('appointments.prescription')->findOrFail($id);
    }

    public function create(array $data): Patient
    {
        return Patient::create($data);
    }

    public function update(Patient $patient, array $data): Patient
    {
        $patient->update($data);
        return $patient;
    }

    public function delete(Patient $patient): void
    {
        $patient->delete(); // soft delete
    }
}
```

app/Repositories/AppointmentRepository.php

```php
```

```php
<?php

namespace App\Repositories;

use App\Models\Appointment;

class AppointmentRepository
{
    public function allForDoctor(int $doctorId, array $filters = [], int $perPa
    {
        return Appointment::with(['patient', 'bookedBy'])
            ->where('doctor_id', $doctorId)
            ->when(isset($filters['date']),   fn($q) => $q->whereDate('appointm
            ->when(isset($filters['status']), fn($q) => $q->where('status', $fi
            ->orderBy('appointment_date')
            ->orderBy('start_time')
            ->paginate($perPage);
    }

    public function allForClinic(int $clinicId, array $filters = [], int $perPa
    {
        return Appointment::with(['patient', 'doctor', 'bookedBy'])
            ->where('clinic_id', $clinicId)
            ->when(isset($filters['date']),   fn($q) => $q->whereDate('appointm
            ->when(isset($filters['status']), fn($q) => $q->where('status', $fi
            ->orderBy('appointment_date')
            ->orderBy('start_time')
            ->paginate($perPage);
    }

    public function allForAdmin(array $filters = [], int $perPage = 15)
    {
        return Appointment::with(['patient', 'doctor', 'clinic'])
            ->when(isset($filters['clinic_id']), fn($q) => $q->where('clinic_id
            ->when(isset($filters['doctor_id']), fn($q) => $q->where('doctor_id
            ->when(isset($filters['status']),    fn($q) => $q->where('status',
            ->when(isset($filters['date']),      fn($q) => $q->whereDate('appoi
            ->orderBy('appointment_date', 'desc')
            ->paginate($perPage);
    }

    public function find(int $id): Appointment
    {
        return Appointment::with(['patient', 'doctor', 'prescription.items'])->
    }
```

```php
    public function create(array $data): Appointment
    {
        return Appointment::create($data);
    }

    public function update(Appointment $appointment, array $data): Appointment
    {
        $appointment->update($data);
        return $appointment;
    }

    public function isSlotTaken(int $doctorId, string $date, string $startTime,
    {
        return Appointment::where('doctor_id', $doctorId)
            ->whereDate('appointment_date', $date)
            ->where('start_time', $startTime)
            ->where('status', '!=', 'cancelled')
            ->when($excludeId, fn($q) => $q->where('id', '!=', $excludeId))
            ->exists();
    }
}
```

app/Repositories/ScheduleRepository.php

```php
```

```php
<?php

namespace App\Repositories;

use App\Models\DoctorSchedule;

class ScheduleRepository
{
    public function allForDoctor(int $doctorId)
    {
        return DoctorSchedule::where('doctor_id', $doctorId)->get();
    }

    public function find(int $id): DoctorSchedule
    {
        return DoctorSchedule::findOrFail($id);
    }

    public function create(array $data): DoctorSchedule
    {
        return DoctorSchedule::create($data);
    }

    public function update(DoctorSchedule $schedule, array $data): DoctorSchedu
    {
        $schedule->update($data);
        return $schedule;
    }

    public function delete(DoctorSchedule $schedule): void
    {
        $schedule->delete();
    }

    public function findForDayAndDoctor(int $doctorId, int $dayOfWeek): ?Doctor
    {
        return DoctorSchedule::where('doctor_id', $doctorId)
            ->where('day_of_week', $dayOfWeek)
            ->where('is_active', true)
            ->first();
    }
}
```

app/Repositories/PrescriptionRepository.php

```php
```

```php
<?php

namespace App\Repositories;

use App\Models\Prescription;

class PrescriptionRepository
{
    public function allForDoctor(int $doctorId, int $perPage = 15)
    {
        return Prescription::with(['patient', 'items'])
            ->where('doctor_id', $doctorId)
            ->latest()
            ->paginate($perPage);
    }

    public function allForAdmin(array $filters = [], int $perPage = 15)
    {
        return Prescription::with(['patient', 'doctor', 'clinic', 'items'])
            ->when(isset($filters['clinic_id']), fn($q) => $q->where('clinic_id
            ->latest()
            ->paginate($perPage);
    }

    public function find(int $id): Prescription
    {
        return Prescription::with(['patient', 'items', 'appointment', 'doctor']
    }

    public function create(array $data, array $items): Prescription
    {
        $prescription = Prescription::create($data);
        $prescription->items()->createMany($items);
        return $prescription->load('items');
    }

    public function update(Prescription $prescription, array $data, array $item
    {
        $prescription->update($data);
        $prescription->items()->delete();
        $prescription->items()->createMany($items);
        return $prescription->load('items');
    }
}
```

Phase 7 — Services

`app/Services/AuthService.php`

> **Passport change:** Token is created with `createToken()->accessToken`. Logout revokes the current OAuth token via `$user->token()->revoke()`.

```php
```

```php
<?php

namespace App\Services;

use App\Models\User;
use Illuminate\Support\Facades\Hash;
use Illuminate\Validation\ValidationException;

class AuthService
{
    /**
     * Validate credentials, check account status, and issue a
     * Passport Personal Access Token.
     */
    public function login(string $email, string $password): array
    {
        $user = User::where('email', $email)->first();

        if (! $user || ! Hash::check($password, $user->password)) {
            throw ValidationException::withMessages([
                'email' => ['The provided credentials are incorrect.'],
            ]);
        }

        if (! $user->is_active) {
            throw ValidationException::withMessages([
                'email' => ['Your account has been deactivated.'],
            ]);
        }

        // createToken() returns PersonalAccessTokenResult.
        // ->accessToken is the plain-text bearer token.
        $token = $user->createToken('clinic-api-token')->accessToken;

        return ['user' => $user, 'token' => $token];
    }

    /**
     * Revoke the current Passport access token.
     * $user->token() returns the token model used in this request.
     */
    public function logout(User $user): void
    {
        $user->token()->revoke();
```

```
    }
}
```

app/Services/AppointmentService.php

```php
```

```php
<?php

namespace App\Services;

use App\Enums\AppointmentStatus;
use App\Models\Appointment;
use App\Repositories\AppointmentRepository;
use App\Repositories\ScheduleRepository;
use Carbon\Carbon;
use Illuminate\Support\Facades\Cache;
use Illuminate\Validation\ValidationException;

class AppointmentService
{
    public function __construct(
        private AppointmentRepository $appointmentRepo,
        private ScheduleRepository $scheduleRepo,
    ) {}

    /**
     * Returns array of available HH:MM slots for a doctor on a given date.
     */
    public function getAvailableSlots(int $doctorId, string $date): array
    {
        $cacheKey = "slots:{$doctorId}:{$date}";

        return Cache::remember($cacheKey, 300, function () use ($doctorId, $dat
            $dayOfWeek = Carbon::parse($date)->dayOfWeek;
            $schedule  = $this->scheduleRepo->findForDayAndDoctor($doctorId, $d

            if (! $schedule) {
                return [];
            }

            $bookedTimes = Appointment::where('doctor_id', $doctorId)
                ->whereDate('appointment_date', $date)
                ->where('status', '!=', 'cancelled')
                ->pluck('start_time')
                ->toArray();

            $slots    = [];
            $current  = Carbon::parse($date . ' ' . $schedule->start_time);
            $end      = Carbon::parse($date . ' ' . $schedule->end_time);
            $duration = $schedule->slot_duration;

            while ($current->copy()->addMinutes($duration)->lte($end)) {
```

```
            $slotTime = $current->format('H:i');
            if (! in_array($slotTime, $bookedTimes)) {
                $slots[] = $slotTime;
            }
            $current->addMinutes($duration);
        }

        return $slots;
    });
}

public function book(array $data): Appointment
{
    $doctorId = $data['doctor_id'];
    $date     = $data['appointment_date'];
    $start    = $data['start_time'];

    $availableSlots = $this->getAvailableSlots($doctorId, $date);

    if (! in_array($start, $availableSlots)) {
        throw ValidationException::withMessages([
            'start_time' => ['This time slot is not available.'],
        ]);
    }

    $dayOfWeek = Carbon::parse($date)->dayOfWeek;
    $schedule  = $this->scheduleRepo->findForDayAndDoctor($doctorId, $dayOf
    $data['end_time'] = Carbon::parse($start)->addMinutes($schedule->slot_d

    Cache::forget("slots:{$doctorId}:{$date}");

    return $this->appointmentRepo->create($data);
}

public function reschedule(Appointment $appointment, array $data): Appointm
{
    if ($appointment->status === AppointmentStatus::Completed) {
        throw ValidationException::withMessages([
            'status' => ['A completed appointment cannot be rescheduled.'],
        ]);
    }

    if ($appointment->status === AppointmentStatus::Cancelled) {
        throw ValidationException::withMessages([
            'status' => ['A cancelled appointment cannot be rescheduled.'],
        ]);
    }
```

```
        $doctorId = $appointment->doctor_id;
        $date     = $data['appointment_date'];
        $start    = $data['start_time'];

        $availableSlots = $this->getAvailableSlots($doctorId, $date);

        if (! in_array($start, $availableSlots)) {
            throw ValidationException::withMessages([
                'start_time' => ['This time slot is not available.'],
            ]);
        }

        $dayOfWeek = Carbon::parse($date)->dayOfWeek;
        $schedule  = $this->scheduleRepo->findForDayAndDoctor($doctorId, $dayOf
        $data['end_time'] = Carbon::parse($start)->addMinutes($schedule->slot_d

        Cache::forget("slots:{$doctorId}:{$appointment->appointment_date->forma
        Cache::forget("slots:{$doctorId}:{$date}");

        return $this->appointmentRepo->update($appointment, $data);
    }

    public function cancel(Appointment $appointment): Appointment
    {
        Cache::forget("slots:{$appointment->doctor_id}:{$appointment->appointme
        return $this->appointmentRepo->update($appointment, ['status' => Appoin
    }

    public function updateStatus(Appointment $appointment, string $status): App
    {
        return $this->appointmentRepo->update($appointment, ['status' => $statu
    }
}
```

app/Services/PrescriptionService.php

```
php
```

```php
<?php

namespace App\Services;

use App\Enums\AppointmentStatus;
use App\Models\Appointment;
use App\Models\Prescription;
use App\Repositories\PrescriptionRepository;
use Illuminate\Validation\ValidationException;

class PrescriptionService
{
    public function __construct(private PrescriptionRepository $repo) {}

    public function create(array $data, array $items, Appointment $appointment,
    {
        if ($appointment->doctor_id !== $doctorId) {
            throw ValidationException::withMessages([
                'appointment_id' => ['You are not the doctor for this appointme
            ]);
        }

        if ($appointment->status === AppointmentStatus::Cancelled) {
            throw ValidationException::withMessages([
                'appointment_id' => ['Cannot create a prescription for a cancel
            ]);
        }

        if ($appointment->prescription()->exists()) {
            throw ValidationException::withMessages([
                'appointment_id' => ['A prescription already exists for this ap
            ]);
        }

        $data['doctor_id']  = $doctorId;
        $data['patient_id'] = $appointment->patient_id;
        $data['clinic_id']  = $appointment->clinic_id;

        return $this->repo->create($data, $items);
    }

    public function update(Prescription $prescription, array $data, array $item
    {
        return $this->repo->update($prescription, $data, $items);
```

```
    }
}
```

app/Services/DashboardService.php *(new)*

```php
<?php

namespace App\Services;

use App\Models\Appointment;
use App\Models\Clinic;
use App\Models\Patient;
use App\Models\Prescription;
use App\Models\User;
use Carbon\Carbon;

class DashboardService
{
    public function summary(?int $clinicId = null): array
    {
        $appointmentQuery  = Appointment::query();
        $patientQuery      = Patient::query();
        $prescriptionQuery = Prescription::query();
        $userQuery         = User::query();

        if ($clinicId) {
            $appointmentQuery->where('clinic_id', $clinicId);
            $patientQuery->where('clinic_id', $clinicId);
            $prescriptionQuery->where('clinic_id', $clinicId);
            $userQuery->where('clinic_id', $clinicId);
        }

        return [
            'total_clinics'           => Clinic::count(),
            'total_users'             => $userQuery->count(),
            'total_patients'          => $patientQuery->count(),
            'total_appointments'      => $appointmentQuery->count(),
            'today_appointments'      => (clone $appointmentQuery)->whereDate('
            'pending_appointments'    => (clone $appointmentQuery)->where('stat
            'total_prescriptions'     => $prescriptionQuery->count(),
        ];
    }
}
```

Phase 8 — Form Requests

app/Http/Requests/Auth/LoginRequest.php

php

```php
<?php

namespace App\Http\Requests\Auth;

use Illuminate\Foundation\Http\FormRequest;

class LoginRequest extends FormRequest
{
    public function authorize(): bool { return true; }

    public function rules(): array
    {
        return [
            'email'    => ['required', 'email'],
            'password' => ['required', 'string'],
        ];
    }
}
```

app/Http/Requests/Clinic/StoreClinicRequest.php

php

```php
<?php

namespace App\Http\Requests\Clinic;

use Illuminate\Foundation\Http\FormRequest;

class StoreClinicRequest extends FormRequest
{
    public function authorize(): bool { return true; }

    public function rules(): array
    {
        return [
            'name'    => ['required', 'string', 'max:255'],
            'address' => ['nullable', 'string'],
            'phone'   => ['nullable', 'string', 'max:20'],
            'email'   => ['nullable', 'email'],
        ];
    }
}
```

app/Http/Requests/Clinic/UpdateClinicRequest.php *(new)*

```php
<?php

namespace App\Http\Requests\Clinic;

use Illuminate\Foundation\Http\FormRequest;

class UpdateClinicRequest extends FormRequest
{
    public function authorize(): bool { return true; }

    public function rules(): array
    {
        return [
            'name'    => ['sometimes', 'string', 'max:255'],
            'address' => ['nullable', 'string'],
            'phone'   => ['nullable', 'string', 'max:20'],
            'email'   => ['nullable', 'email'],
        ];
    }
}
```

app/Http/Requests/User/StoreUserRequest.php

```php
<?php

namespace App\Http\Requests\User;

use App\Enums\UserRole;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;
use Illuminate\Validation\Rules\Password;

class StoreUserRequest extends FormRequest
{
    public function authorize(): bool { return true; }

    public function rules(): array
    {
        return [
            'clinic_id' => ['required', 'exists:clinics,id'],
            'name'      => ['required', 'string', 'max:255'],
            'email'     => ['required', 'email', 'unique:users,email'],
            'password'  => ['required', Password::defaults()],
            'phone'     => ['nullable', 'string', 'max:20'],
            'role'      => ['required', Rule::in([UserRole::Doctor->value, User
        ];
    }
}
```

app/Http/Requests/User/UpdateUserRequest.php *(new)*

```php
```

```php
<?php

namespace App\Http\Requests\User;

use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;
use Illuminate\Validation\Rules\Password;

class UpdateUserRequest extends FormRequest
{
    public function authorize(): bool { return true; }

    public function rules(): array
    {
        return [
            'name'      => ['sometimes', 'string', 'max:255'],
            'email'     => ['sometimes', 'email', Rule::unique('users', 'email'
            'password'  => ['sometimes', Password::defaults()],
            'phone'     => ['nullable', 'string', 'max:20'],
            'clinic_id' => ['sometimes', 'exists:clinics,id'],
        ];
    }
}
```

app/Http/Requests/Patient/StorePatientRequest.php

```php
```

```php
<?php

namespace App\Http\Requests\Patient;

use App\Enums\Gender;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;

class StorePatientRequest extends FormRequest
{
    public function authorize(): bool { return true; }

    public function rules(): array
    {
        return [
            'name'          => ['required', 'string', 'max:255'],
            'phone'         => ['required', 'string', 'max:20'],
            'email'         => ['nullable', 'email'],
            'date_of_birth' => ['nullable', 'date', 'before:today'],
            'gender'        => ['required', Rule::in([Gender::Male->value, Gend
            'address'       => ['nullable', 'string'],
        ];
    }
}
```

app/Http/Requests/Patient/UpdatePatientRequest.php *(new)*

```php
```

```php
<?php

namespace App\Http\Requests\Patient;

use App\Enums\Gender;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;

class UpdatePatientRequest extends FormRequest
{
    public function authorize(): bool { return true; }

    public function rules(): array
    {
        return [
            'name'          => ['sometimes', 'string', 'max:255'],
            'phone'         => ['sometimes', 'string', 'max:20'],
            'email'         => ['nullable', 'email'],
            'date_of_birth' => ['nullable', 'date', 'before:today'],
            'gender'        => ['sometimes', Rule::in([Gender::Male->value, Gen
            'address'       => ['nullable', 'string'],
        ];
    }
}
```

app/Http/Requests/Appointment/StoreAppointmentRequest.php

```php
```

```php
<?php

namespace App\Http\Requests\Appointment;

use Illuminate\Foundation\Http\FormRequest;

class StoreAppointmentRequest extends FormRequest
{
    public function authorize(): bool { return true; }

    public function rules(): array
    {
        return [
            'doctor_id'        => ['required', 'exists:users,id'],
            'patient_id'       => ['required', 'exists:patients,id'],
            'appointment_date' => ['required', 'date', 'after_or_equal:today'],
            'start_time'       => ['required', 'date_format:H:i'],
            'notes'            => ['nullable', 'string'],
        ];
    }
}
```

app/Http/Requests/Appointment/UpdateAppointmentRequest.php ***(new)***

```php
<?php

namespace App\Http\Requests\Appointment;

use Illuminate\Foundation\Http\FormRequest;

class UpdateAppointmentRequest extends FormRequest
{
    public function authorize(): bool { return true; }

    public function rules(): array
    {
        return [
            'appointment_date' => ['required', 'date', 'after_or_equal:today'],
            'start_time'       => ['required', 'date_format:H:i'],
            'notes'            => ['nullable', 'string'],
        ];
    }
}
```

app/Http/Requests/Prescription/StorePrescriptionRequest.php

```php
<?php

namespace App\Http\Requests\Prescription;

use Illuminate\Foundation\Http\FormRequest;

class StorePrescriptionRequest extends FormRequest
{
    public function authorize(): bool { return true; }

    public function rules(): array
    {
        return [
            'appointment_id'         => ['required', 'exists:appointments,id'],
            'diagnosis'              => ['required', 'string'],
            'notes'                  => ['nullable', 'string'],
            'items'                  => ['required', 'array', 'min:1'],
            'items.*.medicine_name'  => ['required', 'string'],
            'items.*.dosage'         => ['required', 'string'],
            'items.*.frequency'      => ['required', 'string'],
            'items.*.duration'       => ['required', 'string'],
            'items.*.notes'          => ['nullable', 'string'],
        ];
    }
}
```

app/Http/Requests/Prescription/UpdatePrescriptionRequest.php *(new)*

```php
```

```php
<?php

namespace App\Http\Requests\Prescription;

use Illuminate\Foundation\Http\FormRequest;

class UpdatePrescriptionRequest extends FormRequest
{
    public function authorize(): bool { return true; }

    public function rules(): array
    {
        return [
            'diagnosis'              => ['sometimes', 'string'],
            'notes'                  => ['nullable', 'string'],
            'items'                  => ['required', 'array', 'min:1'],
            'items.*.medicine_name'  => ['required', 'string'],
            'items.*.dosage'         => ['required', 'string'],
            'items.*.frequency'      => ['required', 'string'],
            'items.*.duration'       => ['required', 'string'],
            'items.*.notes'          => ['nullable', 'string'],
        ];
    }
}
```

app/Http/Requests/Schedule/StoreScheduleRequest.php

```php
```

```php
<?php

namespace App\Http\Requests\Schedule;

use Illuminate\Foundation\Http\FormRequest;

class StoreScheduleRequest extends FormRequest
{
    public function authorize(): bool { return true; }

    public function rules(): array
    {
        return [
            'day_of_week'   => ['required', 'integer', 'between:0,6'],
            'start_time'    => ['required', 'date_format:H:i'],
            'end_time'      => ['required', 'date_format:H:i', 'after:start_tim
            'slot_duration' => ['required', 'integer', 'in:15,20,30,45,60'],
        ];
    }
}
```

app/Http/Requests/Schedule/UpdateScheduleRequest.php *(new)*

```php
<?php

namespace App\Http\Requests\Schedule;

use Illuminate\Foundation\Http\FormRequest;

class UpdateScheduleRequest extends FormRequest
{
    public function authorize(): bool { return true; }

    public function rules(): array
    {
        return [
            'start_time'    => ['sometimes', 'date_format:H:i'],
            'end_time'      => ['sometimes', 'date_format:H:i', 'after:start_ti
            'slot_duration' => ['sometimes', 'integer', 'in:15,20,30,45,60'],
            'is_active'     => ['sometimes', 'boolean'],
        ];
    }
}
```

Phase 9 — API Resources

`app/Http/Resources/ClinicResource.php` *(new)*

```php
<?php

namespace App\Http\Resources;

use Illuminate\Http\Resources\Json\JsonResource;

class ClinicResource extends JsonResource
{
    public function toArray($request): array
    {
        return [
            'id'         => $this->id,
            'name'       => $this->name,
            'address'    => $this->address,
            'phone'      => $this->phone,
            'email'      => $this->email,
            'is_active'  => $this->is_active,
            'created_at' => $this->created_at->toDateTimeString(),
            'users'      => UserResource::collection($this->whenLoaded('users'))
        ];
    }
}
```

`app/Http/Resources/UserResource.php`

```php
```

```php
<?php

namespace App\Http\Resources;

use Illuminate\Http\Resources\Json\JsonResource;

class UserResource extends JsonResource
{
    public function toArray($request): array
    {
        return [
            'id'        => $this->id,
            'name'      => $this->name,
            'email'     => $this->email,
            'phone'     => $this->phone,
            'role'      => $this->role->value,
            'is_active' => $this->is_active,
            'clinic'    => $this->whenLoaded('clinic', fn() => [
                'id'   => $this->clinic->id,
                'name' => $this->clinic->name,
            ]),
        ];
    }
}
```

`app/Http/Resources/PatientResource.php` ***(new)***

```php
```

```php
<?php

namespace App\Http\Resources;

use Illuminate\Http\Resources\Json\JsonResource;

class PatientResource extends JsonResource
{
    public function toArray($request): array
    {
        return [
            'id'            => $this->id,
            'name'          => $this->name,
            'email'         => $this->email,
            'phone'         => $this->phone,
            'gender'        => $this->gender->value,
            'date_of_birth' => $this->date_of_birth?->format('Y-m-d'),
            'address'       => $this->address,
            'clinic_id'     => $this->clinic_id,
            'created_at'    => $this->created_at->toDateTimeString(),
            'appointments'  => AppointmentResource::collection($this->whenLoade
        ];
    }
}
```

app/Http/Resources/ScheduleResource.php ***(new)***

```php
```

```php
<?php

namespace App\Http\Resources;

use Illuminate\Http\Resources\Json\JsonResource;

class ScheduleResource extends JsonResource
{
    public function toArray($request): array
    {
        return [
            'id'            => $this->id,
            'doctor_id'     => $this->doctor_id,
            'clinic_id'     => $this->clinic_id,
            'day_of_week'   => $this->day_of_week,
            'day_name'      => ['Sunday','Monday','Tuesday','Wednesday','Thursd
            'start_time'    => $this->start_time,
            'end_time'      => $this->end_time,
            'slot_duration' => $this->slot_duration,
            'is_active'     => $this->is_active,
        ];
    }
}
```

app/Http/Resources/AppointmentResource.php

```php
```

```php
<?php

namespace App\Http\Resources;

use Illuminate\Http\Resources\Json\JsonResource;

class AppointmentResource extends JsonResource
{
    public function toArray($request): array
    {
        return [
            'id'               => $this->id,
            'appointment_date' => $this->appointment_date->format('Y-m-d'),
            'start_time'       => $this->start_time,
            'end_time'         => $this->end_time,
            'status'           => $this->status->value,
            'notes'            => $this->notes,
            'patient'          => $this->whenLoaded('patient', fn() => new Pati
            'doctor'           => $this->whenLoaded('doctor', fn() => new UserR
            'booked_by'        => $this->whenLoaded('bookedBy', fn() => new Use
            'clinic_id'        => $this->clinic_id,
            'prescription'     => $this->whenLoaded('prescription', fn() =>
                $this->prescription ? new PrescriptionResource($this->prescript
            ),
            'created_at'       => $this->created_at->toDateTimeString(),
        ];
    }
}
```

app/Http/Resources/PrescriptionResource.php

```php
```

```php
<?php

namespace App\Http\Resources;

use Illuminate\Http\Resources\Json\JsonResource;

class PrescriptionResource extends JsonResource
{
    public function toArray($request): array
    {
        return [
            'id'         => $this->id,
            'diagnosis'  => $this->diagnosis,
            'notes'      => $this->notes,
            'created_at' => $this->created_at->toDateTimeString(),
            'doctor'     => $this->whenLoaded('doctor', fn() => new UserResourc
            'patient'    => $this->whenLoaded('patient', fn() => new PatientRes
            'items'      => PrescriptionItemResource::collection($this->whenLoa
        ];
    }
}
```

app/Http/Resources/PrescriptionItemResource.php

```php
```

```php
<?php

namespace App\Http\Resources;

use Illuminate\Http\Resources\Json\JsonResource;

class PrescriptionItemResource extends JsonResource
{
    public function toArray($request): array
    {
        return [
            'id'            => $this->id,
            'medicine_name' => $this->medicine_name,
            'dosage'        => $this->dosage,
            'frequency'     => $this->frequency,
            'duration'      => $this->duration,
            'notes'         => $this->notes,
        ];
    }
}
```

Phase 10 — API Response Helper

app/Traits/ApiResponse.php

```php
```

```php
<?php

namespace App\Traits;

use Illuminate\Http\JsonResponse;
use Illuminate\Http\Resources\Json\ResourceCollection;

trait ApiResponse
{
    protected function success(mixed $data = null, string $message = 'Success',
    {
        $response = ['success' => true, 'message' => $message];

        if ($data !== null) {
            if ($data instanceof ResourceCollection) {
                $paginated = $data->response()->getData(true);
                $response['data'] = $paginated['data'];
                if (isset($paginated['meta'])) {
                    $response['meta'] = $paginated['meta'];
                }
                if (isset($paginated['links'])) {
                    $response['links'] = $paginated['links'];
                }
            } else {
                $response['data'] = $data;
            }
        }

        return response()->json($response, $status);
    }

    protected function error(string $message, int $status = 400, array $errors
    {
        $response = ['success' => false, 'message' => $message];

        if (! empty($errors)) {
            $response['errors'] = $errors;
        }

        return response()->json($response, $status);
    }
}
```

Phase 11 — Controllers

app/Http/Controllers/Auth/AuthController.php

> Uses `auth:api` guard (Passport). Token is a plain-text OAuth bearer token.

```php
```

```php
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use App\Http\Requests\Auth\LoginRequest;
use App\Http\Resources\UserResource;
use App\Services\AuthService;
use App\Traits\ApiResponse;
use Illuminate\Http\Request;

class AuthController extends Controller
{
    use ApiResponse;

    public function __construct(private AuthService $authService) {}

    /**
     * POST /api/auth/login
     * Returns a Passport Personal Access Token.
     * Client must send: Authorization: Bearer <token>
     */
    public function login(LoginRequest $request)
    {
        $result = $this->authService->login($request->email, $request->password

        return $this->success([
            'user'  => new UserResource($result['user']),
            'token' => $result['token'],        // plain-text bearer token
            'token_type' => 'Bearer',
        ], 'Login successful');
    }

    /**
     * POST /api/auth/logout
     * Revokes the current Passport access token.
     */
    public function logout(Request $request)
    {
        $this->authService->logout($request->user());
        return $this->success(message: 'Logged out successfully');
    }

    /**
     * GET /api/auth/me
     */
```

```
    public function me(Request $request)
    {
        return $this->success(new UserResource($request->user()->load('clinic')
    }
}
```

app/Http/Controllers/SuperAdmin/ClinicController.php

```
php
```

```php
<?php

namespace App\Http\Controllers\SuperAdmin;

use App\Http\Controllers\Controller;
use App\Http\Requests\Clinic\StoreClinicRequest;
use App\Http\Requests\Clinic\UpdateClinicRequest;
use App\Http\Resources\ClinicResource;
use App\Models\Clinic;
use App\Repositories\ClinicRepository;
use App\Traits\ApiResponse;

class ClinicController extends Controller
{
    use ApiResponse;

    public function __construct(private ClinicRepository $repo) {}

    public function index()
    {
        return $this->success(ClinicResource::collection($this->repo->all()));
    }

    public function store(StoreClinicRequest $request)
    {
        $clinic = $this->repo->create($request->validated());
        return $this->success(new ClinicResource($clinic), 'Clinic created', 20
    }

    public function show(Clinic $clinic)
    {
        return $this->success(new ClinicResource($this->repo->find($clinic->id)
    }

    public function update(UpdateClinicRequest $request, Clinic $clinic)
    {
        $updated = $this->repo->update($clinic, $request->validated());
        return $this->success(new ClinicResource($updated), 'Clinic updated');
    }

    public function toggle(Clinic $clinic)
    {
        $updated = $this->repo->toggle($clinic);
        return $this->success(new ClinicResource($updated), 'Clinic status togg
```

```
    }
}
```

app/Http/Controllers/SuperAdmin/UserController.php *(new)*

```php
```

```php
<?php

namespace App\Http\Controllers\SuperAdmin;

use App\Http\Controllers\Controller;
use App\Http\Requests\User\StoreUserRequest;
use App\Http\Requests\User\UpdateUserRequest;
use App\Http\Resources\UserResource;
use App\Models\User;
use App\Repositories\UserRepository;
use App\Traits\ApiResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Hash;

class UserController extends Controller
{
    use ApiResponse;

    public function __construct(private UserRepository $repo) {}

    public function index(Request $request)
    {
        $filters = $request->only(['role', 'clinic_id']);
        return $this->success(UserResource::collection($this->repo->all($filter
    }

    public function store(StoreUserRequest $request)
    {
        $data = $request->validated();
        $data['password'] = Hash::make($data['password']);

        $user = $this->repo->create($data);
        return $this->success(new UserResource($user->load('clinic')), 'User cr
    }

    public function show(User $user)
    {
        return $this->success(new UserResource($this->repo->find($user->id)));
    }

    public function update(UpdateUserRequest $request, User $user)
    {
        $data = $request->validated();

        if (isset($data['password'])) {
            $data['password'] = Hash::make($data['password']);
```

```
        }

        $updated = $this->repo->update($user, $data);
        return $this->success(new UserResource($updated->load('clinic')), 'User
    }

    public function toggle(User $user)
    {
        // Prevent deactivating yourself
        abort_if($user->id === request()->user()->id, 422, 'Cannot deactivate y

        $updated = $this->repo->toggle($user);
        return $this->success(new UserResource($updated), 'User status toggled'
    }
}
```

app/Http/Controllers/SuperAdmin/DashboardController.php *(new)*

```
php
```

```php
<?php

namespace App\Http\Controllers\SuperAdmin;

use App\Http\Controllers\Controller;
use App\Http\Resources\AppointmentResource;
use App\Http\Resources\PrescriptionResource;
use App\Repositories\AppointmentRepository;
use App\Repositories\PrescriptionRepository;
use App\Services\DashboardService;
use App\Traits\ApiResponse;
use Illuminate\Http\Request;

class DashboardController extends Controller
{
    use ApiResponse;

    public function __construct(
        private DashboardService $dashboardService,
        private AppointmentRepository $appointmentRepo,
        private PrescriptionRepository $prescriptionRepo,
    ) {}

    /**
     * GET /api/super-admin/dashboard
     * Returns system-wide summary metrics.
     */
    public function index()
    {
        return $this->success($this->dashboardService->summary());
    }

    /**
     * GET /api/super-admin/appointments
     * All appointments across all clinics with optional filters.
     */
    public function appointments(Request $request)
    {
        $filters      = $request->only(['clinic_id', 'doctor_id', 'status', 'da
        $appointments = $this->appointmentRepo->allForAdmin($filters);
        return $this->success(AppointmentResource::collection($appointments));
    }

    /**
     * GET /api/super-admin/prescriptions
     */
```

```
    public function prescriptions(Request $request)
    {
        $filters       = $request->only(['clinic_id']);
        $prescriptions = $this->prescriptionRepo->allForAdmin($filters);
        return $this->success(PrescriptionResource::collection($prescriptions))
    }
}
```

app/Http/Controllers/Doctor/ScheduleController.php ***(new)***

```
php
```

```php
<?php

namespace App\Http\Controllers\Doctor;

use App\Http\Controllers\Controller;
use App\Http\Requests\Schedule\StoreScheduleRequest;
use App\Http\Requests\Schedule\UpdateScheduleRequest;
use App\Http\Resources\ScheduleResource;
use App\Models\DoctorSchedule;
use App\Repositories\ScheduleRepository;
use App\Traits\ApiResponse;
use Illuminate\Http\Request;

class ScheduleController extends Controller
{
    use ApiResponse;

    public function __construct(private ScheduleRepository $repo) {}

    public function index(Request $request)
    {
        $schedules = $this->repo->allForDoctor($request->user()->id);
        return $this->success(ScheduleResource::collection($schedules));
    }

    public function store(StoreScheduleRequest $request)
    {
        $data = array_merge($request->validated(), [
            'doctor_id' => $request->user()->id,
            'clinic_id' => $request->clinic_id,
        ]);

        $schedule = $this->repo->create($data);
        return $this->success(new ScheduleResource($schedule), 'Schedule create
    }

    public function show(Request $request, DoctorSchedule $schedule)
    {
        abort_if($schedule->doctor_id !== $request->user()->id, 403, 'Forbidden
        return $this->success(new ScheduleResource($schedule));
    }

    public function update(UpdateScheduleRequest $request, DoctorSchedule $sche
    {
        abort_if($schedule->doctor_id !== $request->user()->id, 403, 'Forbidden
        $updated = $this->repo->update($schedule, $request->validated());
```

```
        return $this->success(new ScheduleResource($updated), 'Schedule updated
    }

    public function destroy(Request $request, DoctorSchedule $schedule)
    {
        abort_if($schedule->doctor_id !== $request->user()->id, 403, 'Forbidden
        $this->repo->delete($schedule);
        return $this->success(message: 'Schedule deleted');
    }
}
```

app/Http/Controllers/Doctor/AppointmentController.php

```
php
```

```php
<?php

namespace App\Http\Controllers\Doctor;

use App\Http\Controllers\Controller;
use App\Http\Resources\AppointmentResource;
use App\Models\Appointment;
use App\Repositories\AppointmentRepository;
use App\Services\AppointmentService;
use App\Traits\ApiResponse;
use Illuminate\Http\Request;

class AppointmentController extends Controller
{
    use ApiResponse;

    public function __construct(
        private AppointmentRepository $repo,
        private AppointmentService $service,
    ) {}

    public function index(Request $request)
    {
        $appointments = $this->repo->allForDoctor($request->user()->id, $reques
        return $this->success(AppointmentResource::collection($appointments));
    }

    public function show(Request $request, Appointment $appointment)
    {
        abort_if($appointment->doctor_id !== $request->user()->id, 403);
        return $this->success(new AppointmentResource($this->repo->find($appoin
    }

    public function updateStatus(Request $request, Appointment $appointment)
    {
        abort_if($appointment->doctor_id !== $request->user()->id, 403);

        $request->validate([
            'status' => ['required', 'in:confirmed,cancelled,completed'],
        ]);

        $updated = $this->service->updateStatus($appointment, $request->status)
        return $this->success(new AppointmentResource($updated), 'Status update
    }
}
```

app/Http/Controllers/Doctor/PatientController.php *(new)*

```php
<?php

namespace App\Http\Controllers\Doctor;

use App\Http\Controllers\Controller;
use App\Http\Resources\PatientResource;
use App\Repositories\PatientRepository;
use App\Traits\ApiResponse;
use Illuminate\Http\Request;

class PatientController extends Controller
{
    use ApiResponse;

    public function __construct(private PatientRepository $repo) {}

    /**
     * Doctors can list patients from their own clinic (read-only).
     */
    public function index(Request $request)
    {
        $patients = $this->repo->allForClinic(
            $request->clinic_id,
            $request->query('search')
        );
        return $this->success(PatientResource::collection($patients));
    }

    public function show(Request $request, int $id)
    {
        $patient = $this->repo->find($id);
        abort_if($patient->clinic_id !== $request->clinic_id, 403, 'Patient doe
        return $this->success(new PatientResource($patient));
    }
}
```

app/Http/Controllers/Doctor/PrescriptionController.php

```php
```

```php
<?php

namespace App\Http\Controllers\Doctor;

use App\Http\Controllers\Controller;
use App\Http\Requests\Prescription\StorePrescriptionRequest;
use App\Http\Requests\Prescription\UpdatePrescriptionRequest;
use App\Http\Resources\PrescriptionResource;
use App\Models\Appointment;
use App\Models\Prescription;
use App\Repositories\PrescriptionRepository;
use App\Services\PrescriptionService;
use App\Traits\ApiResponse;
use Illuminate\Http\Request;

class PrescriptionController extends Controller
{
    use ApiResponse;

    public function __construct(
        private PrescriptionRepository $repo,
        private PrescriptionService $service,
    ) {}

    public function index(Request $request)
    {
        return $this->success(PrescriptionResource::collection(
            $this->repo->allForDoctor($request->user()->id)
        ));
    }

    public function store(StorePrescriptionRequest $request)
    {
        $appointment  = Appointment::findOrFail($request->appointment_id);
        $prescription = $this->service->create(
            $request->only(['appointment_id', 'diagnosis', 'notes']),
            $request->items,
            $appointment,
            $request->user()->id
        );
        return $this->success(new PrescriptionResource($prescription), 'Prescri
    }

    public function show(Request $request, Prescription $prescription)
    {
        abort_if($prescription->clinic_id !== $request->clinic_id, 403);
```

```
        return $this->success(new PrescriptionResource($this->repo->find($presc
    }

    public function update(UpdatePrescriptionRequest $request, Prescription $pr
    {
        abort_if($prescription->doctor_id !== $request->user()->id, 403);
        $updated = $this->service->update($prescription, $request->only(['diagn
        return $this->success(new PrescriptionResource($updated), 'Prescription
    }
}
```

app/Http/Controllers/Assistant/PatientController.php ***(new)***

```
php
```

```php
<?php

namespace App\Http\Controllers\Assistant;

use App\Http\Controllers\Controller;
use App\Http\Requests\Patient\StorePatientRequest;
use App\Http\Requests\Patient\UpdatePatientRequest;
use App\Http\Resources\PatientResource;
use App\Models\Patient;
use App\Repositories\PatientRepository;
use App\Traits\ApiResponse;
use Illuminate\Http\Request;

class PatientController extends Controller
{
    use ApiResponse;

    public function __construct(private PatientRepository $repo) {}

    public function index(Request $request)
    {
        $patients = $this->repo->allForClinic(
            $request->clinic_id,
            $request->query('search')
        );
        return $this->success(PatientResource::collection($patients));
    }

    public function store(StorePatientRequest $request)
    {
        $data = array_merge($request->validated(), ['clinic_id' => $request->cl
        $patient = $this->repo->create($data);
        return $this->success(new PatientResource($patient), 'Patient created',
    }

    public function show(Request $request, Patient $patient)
    {
        abort_if($patient->clinic_id !== $request->clinic_id, 403, 'Patient doe
        return $this->success(new PatientResource($this->repo->find($patient->i
    }

    public function update(UpdatePatientRequest $request, Patient $patient)
    {
        abort_if($patient->clinic_id !== $request->clinic_id, 403, 'Patient doe
        $updated = $this->repo->update($patient, $request->validated());
        return $this->success(new PatientResource($updated), 'Patient updated')
```

```
    }

    public function destroy(Request $request, Patient $patient)
    {
        abort_if($patient->clinic_id !== $request->clinic_id, 403, 'Patient doe
        $this->repo->delete($patient);
        return $this->success(message: 'Patient deleted');
    }
}
```

app/Http/Controllers/Assistant/AppointmentController.php

```
php
```

```php
<?php

namespace App\Http\Controllers\Assistant;

use App\Http\Controllers\Controller;
use App\Http\Requests\Appointment\StoreAppointmentRequest;
use App\Http\Requests\Appointment\UpdateAppointmentRequest;
use App\Http\Resources\AppointmentResource;
use App\Models\Appointment;
use App\Models\Patient;
use App\Repositories\AppointmentRepository;
use App\Services\AppointmentService;
use App\Traits\ApiResponse;
use Illuminate\Http\Request;

class AppointmentController extends Controller
{
    use ApiResponse;

    public function __construct(
        private AppointmentRepository $repo,
        private AppointmentService $service,
    ) {}

    public function index(Request $request)
    {
        $filters = array_merge(
            $request->only(['status', 'doctor_id']),
            ['date' => $request->query('date', today()->format('Y-m-d'))]
        );
        $appointments = $this->repo->allForClinic($request->clinic_id, $filters
        return $this->success(AppointmentResource::collection($appointments));
    }

    public function store(StoreAppointmentRequest $request)
    {
        $patient = Patient::findOrFail($request->patient_id);
        abort_if($patient->clinic_id !== $request->clinic_id, 422, 'Patient doe

        $data = array_merge($request->validated(), [
            'clinic_id' => $request->clinic_id,
            'booked_by' => $request->user()->id,
        ]);

        $appointment = $this->service->book($data);
        return $this->success(new AppointmentResource($appointment), 'Appointme
```

```
    }

    public function show(Request $request, Appointment $appointment)
    {
        abort_if($appointment->clinic_id !== $request->clinic_id, 403);
        return $this->success(new AppointmentResource($this->repo->find($appoin
    }

    public function update(UpdateAppointmentRequest $request, Appointment $appo
    {
        abort_if($appointment->clinic_id !== $request->clinic_id, 403);
        $updated = $this->service->reschedule($appointment, $request->validated
        return $this->success(new AppointmentResource($updated), 'Appointment r
    }

    public function destroy(Request $request, Appointment $appointment)
    {
        abort_if($appointment->clinic_id !== $request->clinic_id, 403);
        $this->service->cancel($appointment);
        return $this->success(message: 'Appointment cancelled');
    }

    public function availableSlots(Request $request)
    {
        $request->validate([
            'doctor_id' => ['required', 'exists:users,id'],
            'date'      => ['required', 'date', 'after_or_equal:today'],
        ]);

        $slots = $this->service->getAvailableSlots($request->doctor_id, $reques
        return $this->success(['slots' => $slots]);
    }
}
```

Phase 12 — Routes (`routes/api.php`)

> **Passport change:** All `auth:sanctum` guards replaced with `auth:api`.

```
php
```

```php
<?php

use App\Http\Controllers\Auth\AuthController;
use App\Http\Controllers\SuperAdmin\ClinicController;
use App\Http\Controllers\SuperAdmin\UserController;
use App\Http\Controllers\SuperAdmin\DashboardController;
use App\Http\Controllers\Doctor\ScheduleController;
use App\Http\Controllers\Doctor\AppointmentController as DoctorAppointmentContr
use App\Http\Controllers\Doctor\PrescriptionController;
use App\Http\Controllers\Doctor\PatientController as DoctorPatientController;
use App\Http\Controllers\Assistant\PatientController as AssistantPatientControl
use App\Http\Controllers\Assistant\AppointmentController as AssistantAppointmen
use Illuminate\Support\Facades\Route;

// ── Auth ──────────────────────────────────────────────────────────────────
Route::prefix('auth')->group(function () {

    // Public: login (rate-limited to 5 attempts/minute per IP)
    Route::post('login', [AuthController::class, 'login'])
        ->middleware('throttle:5,1');

    // Protected: Passport token guard
    Route::middleware('auth:api')->group(function () {
        Route::post('logout', [AuthController::class, 'logout']);
        Route::get('me',     [AuthController::class, 'me']);
    });
});

// ── Super Admin ───────────────────────────────────────────────────────────
Route::middleware(['auth:api', 'role:super_admin'])
    ->prefix('super-admin')
    ->group(function () {
        Route::apiResource('clinics', ClinicController::class)->except(['destro
        Route::patch('clinics/{clinic}/toggle', [ClinicController::class, 'togg

        Route::apiResource('users', UserController::class)->except(['destroy'])
        Route::patch('users/{user}/toggle', [UserController::class, 'toggle']);

        Route::get('dashboard',     [DashboardController::class, 'index']);
        Route::get('appointments',  [DashboardController::class, 'appointments'
        Route::get('prescriptions', [DashboardController::class, 'prescriptions
    });

// ── Doctor ────────────────────────────────────────────────────────────────
Route::middleware(['auth:api', 'role:doctor', 'clinic.scope'])
    ->prefix('doctor')
```

```
    ->group(function () {
        Route::apiResource('schedules', ScheduleController::class);

        Route::get('appointments',                        [DoctorAppointmentCon
        Route::get('appointments/{appointment}',          [DoctorAppointmentCon
        Route::patch('appointments/{appointment}/status', [DoctorAppointmentCon

        Route::apiResource('prescriptions', PrescriptionController::class)->exc

        Route::get('patients',           [DoctorPatientController::class, 'inde
        Route::get('patients/{patient}', [DoctorPatientController::class, 'show
    });

// ── Assistant ──────────────────────────────────────────────────────────────
Route::middleware(['auth:api', 'role:assistant', 'clinic.scope'])
    ->prefix('assistant')
    ->group(function () {
        Route::apiResource('patients', AssistantPatientController::class);

        Route::get('appointments',                    [AssistantAppointmentCon
        Route::post('appointments',                   [AssistantAppointmentCon
        Route::get('appointments/{appointment}',      [AssistantAppointmentCon
        Route::put('appointments/{appointment}',      [AssistantAppointmentCon
        Route::delete('appointments/{appointment}',   [AssistantAppointmentCon
        Route::get('available-slots',                 [AssistantAppointmentCon
    });
```

Phase 13 — Seeders

`database/seeders/SuperAdminSeeder.php`

```
php
```

```php
<?php

namespace Database\Seeders;

use App\Enums\UserRole;
use App\Models\User;
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\Hash;

class SuperAdminSeeder extends Seeder
{
    public function run(): void
    {
        User::firstOrCreate(
            ['email' => 'admin@clinic.com'],
            [
                'name'      => 'Super Admin',
                'password'  => Hash::make('password'),
                'role'      => UserRole::SuperAdmin,
                'is_active' => true,
            ]
        );
    }
}
```

database/seeders/ClinicSeeder.php

```php
```

```php
<?php

namespace Database\Seeders;

use App\Models\Clinic;
use Illuminate\Database\Seeder;

class ClinicSeeder extends Seeder
{
    public function run(): void
    {
        Clinic::insert([
            ['name' => 'Cairo Clinic',      'phone' => '0201111111', 'email' =>
            ['name' => 'Alexandria Clinic', 'phone' => '0203333333', 'email' =>
        ]);
    }
}
```

`database/seeders/DatabaseSeeder.php` *(new — wires everything together)*

```php
<?php

namespace Database\Seeders;

use Illuminate\Database\Seeder;

class DatabaseSeeder extends Seeder
{
    public function run(): void
    {
        $this->call([
            ClinicSeeder::class,
            SuperAdminSeeder::class,
        ]);
    }
}
```

Phase 14 — Security Hardening

Exception handler in `bootstrap/app.php`

```php
```

```
->withExceptions(function (Exceptions $exceptions) {
    $exceptions->render(function (\Illuminate\Validation\ValidationException $e
        if ($request->expectsJson()) {
            return response()->json([
                'success' => false,
                'message' => 'Validation failed',
                'errors'  => $e->errors(),
            ], 422);
        }
    });

    $exceptions->render(function (\Illuminate\Auth\AuthenticationException $e,
        if ($request->expectsJson()) {
            return response()->json(['success' => false, 'message' => 'Unauthen
        }
    });

    $exceptions->render(function (\Symfony\Component\HttpKernel\Exception\Acces
        if ($request->expectsJson()) {
            return response()->json(['success' => false, 'message' => 'Forbidde
        }
    });

    $exceptions->render(function (\Illuminate\Database\Eloquent\ModelNotFoundEx
        if ($request->expectsJson()) {
            return response()->json(['success' => false, 'message' => 'Resource
        }
    });
})
```

Clinic-scope enforcement reminder

Every repository method that fetches clinic-scoped data must include a `where('clinic_id', ...)` guard. **Never trust a client-supplied `clinic_id`** — always use `$request->clinic_id` which was injected by `ClinicScopeMiddleware` from `$user->clinic_id`.

Phase 15 — Model Factories *(new — required for tests)*

`database/factories/ClinicFactory.php`

```
php
```

```php
<?php

namespace Database\Factories;

use Illuminate\Database\Eloquent\Factories\Factory;

class ClinicFactory extends Factory
{
    public function definition(): array
    {
        return [
            'name'      => $this->faker->company(),
            'address'   => $this->faker->address(),
            'phone'     => $this->faker->phoneNumber(),
            'email'     => $this->faker->unique()->companyEmail(),
            'is_active' => true,
        ];
    }

    public function inactive(): static
    {
        return $this->state(['is_active' => false]);
    }
}
```

`database/factories/UserFactory.php` ***(updated)***

```php
```

```php
<?php

namespace Database\Factories;

use App\Enums\UserRole;
use Illuminate\Database\Eloquent\Factories\Factory;
use Illuminate\Support\Facades\Hash;

class UserFactory extends Factory
{
    public function definition(): array
    {
        return [
            'clinic_id' => null,
            'name'      => $this->faker->name(),
            'email'     => $this->faker->unique()->safeEmail(),
            'password'  => Hash::make('password'),
            'phone'     => $this->faker->phoneNumber(),
            'role'      => UserRole::Doctor,
            'is_active' => true,
        ];
    }

    public function superAdmin(): static
    {
        return $this->state(['role' => UserRole::SuperAdmin, 'clinic_id' => nul
    }

    public function doctor(): static
    {
        return $this->state(['role' => UserRole::Doctor]);
    }

    public function assistant(): static
    {
        return $this->state(['role' => UserRole::Assistant]);
    }

    public function inactive(): static
    {
        return $this->state(['is_active' => false]);
    }
}
```

database/factories/PatientFactory.php

```php
<?php

namespace Database\Factories;

use App\Enums\Gender;
use App\Models\Clinic;
use Illuminate\Database\Eloquent\Factories\Factory;

class PatientFactory extends Factory
{
    public function definition(): array
    {
        return [
            'clinic_id'     => Clinic::factory(),
            'name'          => $this->faker->name(),
            'email'         => $this->faker->optional()->safeEmail(),
            'phone'         => $this->faker->phoneNumber(),
            'date_of_birth' => $this->faker->date('Y-m-d', '-18 years'),
            'gender'        => $this->faker->randomElement([Gender::Male->value
            'address'       => $this->faker->optional()->address(),
        ];
    }
}
```

database/factories/DoctorScheduleFactory.php

```php
```

```php
<?php

namespace Database\Factories;

use App\Models\Clinic;
use App\Models\User;
use Illuminate\Database\Eloquent\Factories\Factory;

class DoctorScheduleFactory extends Factory
{
    public function definition(): array
    {
        return [
            'doctor_id'     => User::factory()->doctor(),
            'clinic_id'     => Clinic::factory(),
            'day_of_week'   => $this->faker->numberBetween(0, 6),
            'start_time'    => '09:00',
            'end_time'      => '17:00',
            'slot_duration' => 30,
            'is_active'     => true,
        ];
    }
}
```

database/factories/AppointmentFactory.php

```php
```

```php
<?php

namespace Database\Factories;

use App\Enums\AppointmentStatus;
use App\Models\Clinic;
use App\Models\Patient;
use App\Models\User;
use Illuminate\Database\Eloquent\Factories\Factory;

class AppointmentFactory extends Factory
{
    public function definition(): array
    {
        return [
            'clinic_id'        => Clinic::factory(),
            'doctor_id'        => User::factory()->doctor(),
            'patient_id'       => Patient::factory(),
            'booked_by'        => User::factory()->assistant(),
            'appointment_date' => $this->faker->dateTimeBetween('now', '+1 mont
            'start_time'       => '09:00',
            'end_time'         => '09:30',
            'status'           => AppointmentStatus::Pending,
            'notes'            => null,
        ];
    }
}
```

Phase 16 — Docker

Project directory structure

```
clinic-api/
├── docker/
│   ├── nginx/
│   │   └── default.conf
│   ├── php/
│   │   └── php.ini
│   └── entrypoint.sh
├── Dockerfile
├── docker-compose.yml
└── .dockerignore
```

Dockerfile

dockerfile

```
# ─── Build stage ──────────────────────────────────────────────────────
FROM php:8.3-fpm-alpine AS base

# System dependencies
RUN apk add --no-cache \
    bash \
    curl \
    libpng-dev \
    libjpeg-turbo-dev \
    libwebp-dev \
    libzip-dev \
    oniguruma-dev \
    openssl-dev \
    zip \
    unzip \
    git \
    redis

# PHP extensions
RUN docker-php-ext-configure gd --with-jpeg --with-webp \
 && docker-php-ext-install -j$(nproc) \
    gd \
    mbstring \
    pdo_mysql \
    pcntl \
    bcmath \
    zip \
    opcache

# Redis PHP extension (via PECL)
RUN pecl install redis && docker-php-ext-enable redis

# Composer
COPY --from=composer:2.7 /usr/bin/composer /usr/bin/composer

WORKDIR /var/www/html

# Copy composer files first (cache layer)
COPY composer.json composer.lock ./
RUN composer install --no-dev --no-scripts --no-autoloader --prefer-dist

# Copy application
COPY . .

# Finish composer install
RUN composer dump-autoload --optimize
```

```
# Set permissions
RUN chown -R www-data:www-data storage bootstrap/cache \
 && chmod -R 775 storage bootstrap/cache

# Copy custom php.ini
COPY docker/php/php.ini $PHP_INI_DIR/conf.d/custom.ini

# Entrypoint script
COPY docker/entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh

EXPOSE 9000

ENTRYPOINT ["/entrypoint.sh"]
CMD ["php-fpm"]
```

---

docker/entrypoint.sh

bash

```bash
#!/bin/bash
set -e

echo "==> Waiting for MySQL to be ready..."
until php -r "new PDO('mysql:host=${DB_HOST};port=${DB_PORT}', '${DB_USERNAME}'
    echo "    MySQL not ready yet, retrying in 3s..."
    sleep 3
done

echo "==> Running migrations..."
php artisan migrate --force

echo "==> Caching config and routes..."
php artisan config:cache
php artisan route:cache
php artisan event:cache

# Install Passport keys if not already present
if [ ! -f storage/oauth-private.key ]; then
    echo "==> Installing Passport keys..."
    php artisan passport:install --force
fi

echo "==> Starting PHP-FPM..."
exec "$@"
```

---

docker/nginx/default.conf

nginx

```
server {
    listen 80;
    server_name _;

    root /var/www/html/public;
    index index.php;

    client_max_body_size 20M;

    # Gzip
    gzip on;
    gzip_types text/plain application/json application/javascript text/css;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location ~ \.php$ {
        fastcgi_pass   app:9000;
        fastcgi_index  index.php;
        fastcgi_param  SCRIPT_FILENAME $document_root$fastcgi_script_name;
        include        fastcgi_params;
        fastcgi_read_timeout 300;
    }

    location ~ /\.(?!well-known).* {
        deny all;
    }

    location = /favicon.ico { access_log off; log_not_found off; }
    location = /robots.txt  { access_log off; log_not_found off; }

    error_log  /var/log/nginx/error.log;
    access_log /var/log/nginx/access.log;
}
```

---

docker/php/php.ini

ini

```
; Performance
opcache.enable=1
opcache.memory_consumption=256
opcache.interned_strings_buffer=16
opcache.max_accelerated_files=20000
opcache.validate_timestamps=0

; Limits
memory_limit=256M
upload_max_filesize=20M
post_max_size=20M
max_execution_time=120

; Error reporting (disable in production)
display_errors=Off
log_errors=On
error_log=/var/log/php_errors.log
```

---

docker-compose.yml

yaml

```yaml
version: "3.9"

services:

  # ── PHP-FPM Application ──────────────────────────────────────
  app:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: clinic_app
    restart: unless-stopped
    volumes:
      - .:/var/www/html
      - ./storage:/var/www/html/storage   # persist logs & uploads
    environment:
      APP_ENV: ${APP_ENV:-local}
      APP_KEY: ${APP_KEY}
      DB_HOST: db
      DB_PORT: 3306
      DB_DATABASE: ${DB_DATABASE:-clinic_api}
      DB_USERNAME: ${DB_USERNAME:-clinic_user}
      DB_PASSWORD: ${DB_PASSWORD:-secret}
      REDIS_HOST: redis
      CACHE_DRIVER: redis
      QUEUE_CONNECTION: redis
      SESSION_DRIVER: redis
      PASSPORT_PERSONAL_ACCESS_CLIENT_ID: ${PASSPORT_PERSONAL_ACCESS_CLIENT_ID}
      PASSPORT_PERSONAL_ACCESS_CLIENT_SECRET: ${PASSPORT_PERSONAL_ACCESS_CLIENT
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_started
    networks:
      - clinic_net

  # ── Nginx ────────────────────────────────────────────────────
  nginx:
    image: nginx:1.25-alpine
    container_name: clinic_nginx
    restart: unless-stopped
    ports:
      - "8000:80"
    volumes:
      - .:/var/www/html
      - ./docker/nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
```

```yaml
    depends_on:
      - app
    networks:
      - clinic_net

  # ── MySQL ──────────────────────────────────────────────────────────────
  db:
    image: mysql:8.0
    container_name: clinic_db
    restart: unless-stopped
    environment:
      MYSQL_DATABASE: ${DB_DATABASE:-clinic_api}
      MYSQL_USER: ${DB_USERNAME:-clinic_user}
      MYSQL_PASSWORD: ${DB_PASSWORD:-secret}
      MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD:-rootsecret}
    ports:
      - "3307:3306"    # 3307 on host avoids conflicts with a local MySQL
    volumes:
      - clinic_db_data:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "root", "-p$
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - clinic_net

  # ── Redis ──────────────────────────────────────────────────────────────
  redis:
    image: redis:7-alpine
    container_name: clinic_redis
    restart: unless-stopped
    command: redis-server --maxmemory 256mb --maxmemory-policy allkeys-lru
    ports:
      - "6380:6379"    # 6380 on host avoids conflicts with a local Redis
    volumes:
      - clinic_redis_data:/data
    networks:
      - clinic_net

  # ── Queue Worker ───────────────────────────────────────────────────────
  worker:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: clinic_worker
    restart: unless-stopped
```

```
    command: php artisan queue:work redis --sleep=3 --tries=3 --max-time=3600
    volumes:
      - .:/var/www/html
    environment:
      APP_ENV: ${APP_ENV:-local}
      APP_KEY: ${APP_KEY}
      DB_HOST: db
      DB_DATABASE: ${DB_DATABASE:-clinic_api}
      DB_USERNAME: ${DB_USERNAME:-clinic_user}
      DB_PASSWORD: ${DB_PASSWORD:-secret}
      REDIS_HOST: redis
      QUEUE_CONNECTION: redis
    depends_on:
      - app
      - db
      - redis
    networks:
      - clinic_net

volumes:
  clinic_db_data:
  clinic_redis_data:

networks:
  clinic_net:
    driver: bridge
```

.dockerignore

```
.git
.github
.env
.env.*
node_modules
vendor
storage/logs/*
storage/framework/cache/*
storage/framework/sessions/*
storage/framework/views/*
bootstrap/cache/*
*.log
docker-compose.override.yml
```

Phase 17 — Testing

> **Passport note for tests:** Call `Passport::actingAs($user)` (or simply `$this->actingAs($user, 'api')`) in feature tests — both work. You do **not** need a real token in the test environment.

Add to `tests/TestCase.php` if using `actingAs` consistently:

```php
use Laravel\Passport\Passport;

// Inside your test:
Passport::actingAs($user);
// or:
$this->actingAs($user, 'api');
```

`tests/Feature/AuthTest.php`

```php
```

```php
<?php

namespace Tests\Feature;

use App\Models\User;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Laravel\Passport\Passport;
use Tests\TestCase;

class AuthTest extends TestCase
{
    use RefreshDatabase;

    public function test_user_can_login_with_valid_credentials(): void
    {
        $user = User::factory()->create(['password' => bcrypt('secret')]);

        $this->postJson('/api/auth/login', ['email' => $user->email, 'password'
            ->assertOk()
            ->assertJsonStructure(['data' => ['token', 'token_type', 'user']]);
    }

    public function test_login_fails_with_wrong_password(): void
    {
        $user = User::factory()->create();

        $this->postJson('/api/auth/login', ['email' => $user->email, 'password'
            ->assertStatus(422);
    }

    public function test_inactive_user_cannot_login(): void
    {
        $user = User::factory()->inactive()->create(['password' => bcrypt('secr

        $this->postJson('/api/auth/login', ['email' => $user->email, 'password'
            ->assertStatus(422);
    }

    public function test_authenticated_user_can_fetch_profile(): void
    {
        $user = User::factory()->create();
        Passport::actingAs($user);

        $this->getJson('/api/auth/me')
            ->assertOk()
            ->assertJsonPath('data.email', $user->email);
```

```
    }

    public function test_authenticated_user_can_logout(): void
    {
        $user = User::factory()->create();
        Passport::actingAs($user);

        $this->postJson('/api/auth/logout')->assertOk();
    }
}
```

tests/Feature/RbacTest.php

```
php
```

```php
<?php

namespace Tests\Feature;

use App\Enums\UserRole;
use App\Models\Clinic;
use App\Models\User;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Laravel\Passport\Passport;
use Tests\TestCase;

class RbacTest extends TestCase
{
    use RefreshDatabase;

    public function test_doctor_cannot_access_super_admin_routes(): void
    {
        $clinic = Clinic::factory()->create();
        $doctor = User::factory()->doctor()->create(['clinic_id' => $clinic->id
        Passport::actingAs($doctor);

        $this->getJson('/api/super-admin/clinics')->assertForbidden();
    }

    public function test_assistant_cannot_access_doctor_routes(): void
    {
        $clinic    = Clinic::factory()->create();
        $assistant = User::factory()->assistant()->create(['clinic_id' => $clin
        Passport::actingAs($assistant);

        $this->getJson('/api/doctor/schedules')->assertForbidden();
    }

    public function test_super_admin_can_access_all_clinics(): void
    {
        $admin = User::factory()->superAdmin()->create();
        Clinic::factory()->count(3)->create();
        Passport::actingAs($admin);

        $this->getJson('/api/super-admin/clinics')
            ->assertOk()
            ->assertJsonStructure(['data']);
    }

    public function test_unauthenticated_request_returns_401(): void
    {
```

```
        $this->getJson('/api/auth/me')->assertUnauthorized();
    }
}
```

tests/Feature/ClinicScopeTest.php

```php
```

```php
<?php

namespace Tests\Feature;

use App\Models\Clinic;
use App\Models\Patient;
use App\Models\User;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Laravel\Passport\Passport;
use Tests\TestCase;

class ClinicScopeTest extends TestCase
{
    use RefreshDatabase;

    public function test_doctor_cannot_see_patients_from_another_clinic(): void
    {
        $clinic1 = Clinic::factory()->create();
        $clinic2 = Clinic::factory()->create();

        $doctor = User::factory()->doctor()->create(['clinic_id' => $clinic1->i
        Patient::factory()->create(['clinic_id' => $clinic2->id]);

        Passport::actingAs($doctor);

        $response = $this->getJson('/api/doctor/patients');
        $response->assertOk();
        $this->assertCount(0, $response->json('data'));
    }

    public function test_inactive_clinic_blocks_staff(): void
    {
        $clinic = Clinic::factory()->inactive()->create();
        $doctor = User::factory()->doctor()->create(['clinic_id' => $clinic->id
        Passport::actingAs($doctor);

        $this->getJson('/api/doctor/patients')->assertForbidden();
    }
}
```

tests/Feature/AppointmentTest.php

```php
```

```php
<?php

namespace Tests\Feature;

use App\Models\Clinic;
use App\Models\DoctorSchedule;
use App\Models\Patient;
use App\Models\User;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Laravel\Passport\Passport;
use Tests\TestCase;

class AppointmentTest extends TestCase
{
    use RefreshDatabase;

    private function setupClinic(): array
    {
        $clinic    = Clinic::factory()->create();
        $doctor    = User::factory()->doctor()->create(['clinic_id' => $clinic-
        $assistant = User::factory()->assistant()->create(['clinic_id' => $clin
        $patient   = Patient::factory()->create(['clinic_id' => $clinic->id]);

        DoctorSchedule::factory()->create([
            'doctor_id'     => $doctor->id,
            'clinic_id'     => $clinic->id,
            'day_of_week'   => now()->next('Monday')->dayOfWeek,
            'start_time'    => '09:00',
            'end_time'      => '12:00',
            'slot_duration' => 30,
            'is_active'     => true,
        ]);

        return compact('clinic', 'doctor', 'assistant', 'patient');
    }

    public function test_double_booking_is_prevented(): void
    {
        ['doctor' => $doctor, 'assistant' => $assistant, 'patient' => $patient]
        Passport::actingAs($assistant);

        $date    = now()->next('Monday')->format('Y-m-d');
        $payload = [
            'doctor_id'        => $doctor->id,
            'patient_id'       => $patient->id,
            'appointment_date' => $date,
```

```
            'start_time'       => '09:00',
        ];

        $this->postJson('/api/assistant/appointments', $payload)->assertStatus(
        $this->postJson('/api/assistant/appointments', $payload)->assertStatus(
    }

    public function test_patient_from_other_clinic_cannot_be_booked(): void
    {
        ['assistant' => $assistant, 'doctor' => $doctor] = $this->setupClinic()
        $otherPatient = Patient::factory()->create(); // different clinic
        Passport::actingAs($assistant);

        $this->postJson('/api/assistant/appointments', [
            'doctor_id'        => $doctor->id,
            'patient_id'       => $otherPatient->id,
            'appointment_date' => now()->next('Monday')->format('Y-m-d'),
            'start_time'       => '09:00',
        ])->assertStatus(422);
    }

    public function test_available_slots_returns_correct_slots(): void
    {
        ['doctor' => $doctor, 'assistant' => $assistant] = $this->setupClinic()
        Passport::actingAs($assistant);

        $date = now()->next('Monday')->format('Y-m-d');

        $response = $this->getJson("/api/assistant/available-slots?doctor_id={$
        $response->assertOk();
        $this->assertNotEmpty($response->json('data.slots'));
    }
}
```

Phase 18 — Running Locally vs Docker

Running with Docker (recommended)

bash

```
# 1. Copy environment file
cp .env.example .env

# 2. Fill in .env — APP_KEY is generated below; Passport keys are generated by
php artisan key:generate --show   # copy output into APP_KEY in .env

# 3. Build and start all services
docker compose up -d --build

# 4. Seed the database (runs migrations + seeders inside the container)
docker compose exec app php artisan db:seed

# 5. Run tests inside Docker
docker compose exec app php artisan test

# 6. Access the API
# http://localhost:8000/api/auth/login
```

Useful Docker commands

bash

```
# View running containers
docker compose ps

# Tail app logs
docker compose logs -f app

# Shell into the app container
docker compose exec app bash

# Restart worker after code change
docker compose restart worker

# Stop everything and remove volumes (fresh start)
docker compose down -v
```

Running locally (without Docker)

bash

```
php artisan migrate
php artisan passport:install
php artisan db:seed
php artisan serve
php artisan queue:work redis
php artisan test
./vendor/bin/pint
```

Quick Reference: Business Rules Enforcement Map

Rule	Enforced In
BR-01 / BR-08	`ClinicScopeMiddleware` + repository `where clinic_id`
BR-02	`ClinicScopeMiddleware` (bypasses for super_admin)
BR-03	`AppointmentService::book()` — slot availability check
BR-04	`PrescriptionService::create()` — doctor_id match
BR-05	`AppointmentService::reschedule()` — status check (completed/cancelled)
BR-06	`PrescriptionService::create()` — status !== cancelled
BR-07	`AssistantAppointmentController::store()` — clinic_id match
BR-08	`PrescriptionService::create()` — duplicate prescription guard
BR-09	`AppointmentService` (app level) + DB unique constraint

What Changed vs the Original Guide

Area	Original	Updated
Auth package	`laravel/sanctum`	`laravel/passport`
Token guard	`auth:sanctum`	`auth:api`
User model trait	`Laravel\Sanctum\HasApiTokens`	`Laravel\Passport\`
Token creation	`createToken()->plainTextToken`	`createToken()->ac`

Area	Original	Updated
Logout	`currentAccessToken()->delete()`	`token()->revoke()`
`ClinicResource`	Missing	Added
`PatientResource`	Missing	Added
`ScheduleResource`	Missing	Added
`UpdateClinicRequest`	Missing	Added
`UpdateUserRequest`	Missing	Added
`UpdatePatientRequest`	Missing	Added
`UpdateAppointmentRequest`	Missing	Added
`UpdatePrescriptionRequest`	Missing	Added
`UpdateScheduleRequest`	Missing	Added
`UserController` (SuperAdmin)	Missing	Added
`DashboardController`	Missing	Added
`DashboardService`	Missing	Added
`ScheduleController`	Missing	Added
`DoctorPatientController`	Missing	Added
`AssistantPatientController`	Missing	Added
Model Factories	Missing	Added (all 5)
`DatabaseSeeder`	Missing	Added
Docker	Missing	Added (Dockerfile, com entrypoint)
Passport duplicate prescription guard	Missing	Added to `Prescripti`
Cancelled-appointment reschedule guard	Missing	Added to `Appointmen`